Série théorique : PHP procédural

GET / POST – define – include – die – echo – chaînes – tableaux – dates – MySQLi – formulaires

4ème SI – Lycée Rafèha – Enseignant : Aymen Essouyah

Sommaire

1. Premiers pas : balises, variables et echo . . . 2
2. Les chaînes de caractères : différence entre ' et " . . . 3
3. Les constantes : define . . . 6
4. Réutiliser du code : include et require . . . 6
5. Arrêter le script : die et exit . . . 8
6. Récupérer les données : GET et POST . . . 8
7. Types de données et conversion . . . 10
8. Conditions et boucles . . . 11
9. Tableaux et tableaux associatifs . . . 13
10. Les dates . . . 15
11. MySQLi procédural : relier PHP à la base de données . . . 19
12. Formulaires interactifs : boutons radio et calcul d'âge . . . 22
13. Pour s'exercer . . . 30
Annexe A – Les types de données PHP . . . 31
Annexe B – Les formats de la fonction date() . . . 32
Annexe C – Récapitulatif MySQLi procédural . . . 34
Annexe D – Messages d'erreur fréquents . . . 35

Avant de commencer

- Tous les programmes de cette série sont écrits en **PHP procédural** : des fonctions. L'accès à MySQL se fait uniquement avec les fonctions `mysqli_...` .
- Chaque exemple a été exécuté pour de vrai (PHP 8.3, MariaDB 10.11) : les cadres « Affichage » montrent le résultat réel. Les exemples MySQLi utilisent la base **Dar Zitouna** de la série 8 (tables `SERVEUR` et `COMMANDE`, avec les colonnes `date_embauche` et `date_cmd`).
- Les pages PHP s'ouvrent par le serveur : fichier placé dans `htdocs`, adresse `http://localhost/dossier/page.php`. Un double-clic sur le fichier n'exécute pas le PHP.
- Conventions : un cadre gris = code ; un cadre à bord noir = affichage ; « Piège » = erreur fréquente à connaître.

1. Premiers pas : balises, variables et echo

Un fichier `.php` mélange du HTML et du PHP. Le serveur exécute uniquement ce qui est entre `<?php` et `?>` et envoie au navigateur du HTML pur : le visiteur ne voit jamais le code PHP.

- Chaque instruction se termine par `;`. Commentaires : `// ligne`, `# ligne`, `/* bloc */`.
- Une variable commence par `$`, ne se déclare pas, et PHP distingue majuscules et minuscules (`$nom` ≠ `$Nom`).
- Un nom de variable commence par une lettre ou _, jamais par un chiffre.

1.1 echo : afficher du texte et des variables

Fichier : echo1.php

```
<?php
$nom = "Sami";
$age = 17;
echo "Bonjour " . $nom . "<br>";
echo "Vous avez ", $age, " ans<br>";
echo "Dans 3 ans : ", $age + 3, " ans";
?>
```

Affichage obtenu

```
Bonjour Sami<br>Vous avez 17 ans<br>Dans 3 ans : 20 ans
```

`echo` accepte **plusieurs arguments séparés par des virgules** ; le point `.` **concatène** (colle) des chaînes. Le résultat affiché est le même, mais la virgule ne fabrique pas de chaîne. `print` fait presque la même chose mais n'accepte qu'un argument. `<?= $x ?>` est une écriture courte de `<?php echo $x; ?>`.

Fichier : echo2.php

```
<?php
$prix = 12.5;
echo "Prix : ", $prix, " DT\n";
print "Prix : " . $prix . " DT\n";
?>
<p>Prix : <?= $prix ?> DT</p>
```

Affichage obtenu

```
Prix : 12.5 DT
Prix : 12.5 DT
<p>Prix : 12.5 DT</p>
```

1.2 Sauts de ligne : \n ou
 ?

Écriture	Effet	Où le voit-on ?
`"\n"` (entre guillemets doubles)	saut de ligne dans le **code source** de la page	Affichage > Code source ; invisible dans la page
`" "`	saut de ligne dans la **page** affichée	dans le navigateur
`"\t"`	tabulation (dans le code source)	invisible dans la page
`echo "a"; echo "b";`	affiche ab collé (aucun espace automatique)	dans la page

À retenir

- `echo` : affiche ; virgule = plusieurs arguments, point = concaténation.
- Pour aller à la ligne dans la page : `<br>` ou une balise `<p>` ; `\n` ne se voit que dans le code source.
- `echo` n'ajoute ni espace ni saut de ligne : c'est à vous de les écrire.

Piège : `\n` n'est interprété qu'entre **guillemets doubles**. Dans `'ligne\n'` (guillemets simples), le navigateur reçoit les deux caractères `\` et `n`.

2. Les chaînes de caractères : différence entre ' et "

Une chaîne s'écrit entre guillemets simples ' ' ou doubles " ". Ce n'est **pas** équivalent :

	Guillemets simples ' '	**Guillemets doubles " "**
Variables	**non** remplacées : `$nom` reste `$nom`	remplacées par leur valeur
Séquences `\n` `\t`	non interprétées (affichées telles quelles)	interprétées (saut de ligne, tabulation)
Caractères à échapper	`\'` (apostrophe) et `\\`	`\"` `\$` `\\`
À utiliser pour	texte fixe ; HTML dont les attributs sont entre "	texte contenant des variables ; HTML dont les attributs sont entre '

Fichier : quotes1.php

```
<?php
$nom = "Sami";
echo 'Bonjour $nom\n';
echo "\n";
echo "Bonjour $nom\n";
echo 'L\'été';
echo "\n";
echo "Il dit \"salut\"\n";
echo "Prix : \$5\n";
echo 'Dossier C:\\tmp';
?>
```

Affichage obtenu

```
Bonjour $nom\n
Bonjour Sami
L'été
Il dit "salut"
Prix : $5
Dossier C:\tmp
```

2.1 Choisir les bons guillemets pour écrire du HTML

La règle pratique : les guillemets de PHP doivent être **différents** de ceux du HTML écrit à l'intérieur. Sinon, il faut échapper avec `\`.

Fichier : quotes2.php

```
<?php
$nom = "Sami";
echo "<input type='text' name='nom' value='$nom'>\n";
echo '<input type="text" name="nom" value="' . $nom . '">' . "\n";
echo "<input type=\"text\" name=\"nom\" value=\"$nom\">\n";
?>
```

Affichage obtenu

```
<input type='text' name='nom' value='Sami'>
<input type="text" name="nom" value="Sami">
<input type="text" name="nom" value="Sami">
```

2.2 Un élément de tableau dans une chaîne

Fichier : quotes3.php

```
<?php
$s = ["nom" => "Rim", "salle" => "Salon"];
echo "Salle : $s[salle]\n";
echo "Serveur : {$s['nom']}\n";
echo 'Serveur : ' . $s['nom'] . "\n";
?>
```

Affichage obtenu

```
Salle : Salon
Serveur : Rim
Serveur : Rim
```

Entre guillemets doubles, on écrit `$s[salle]` **sans** apostrophes, ou bien `{$s['nom']}` **avec** accolades. Écrire `"$s['nom']"` provoque une erreur de syntaxe.

2.3 Le cas de ''' : n'existe pas en PHP

Les triples guillemets `'''` (ou `"""`) du langage Python **n'existent pas** en PHP. Pour écrire un texte sur plusieurs lignes, on utilise soit une chaîne normale qui contient des retours à la ligne, soit la syntaxe **heredoc** `<<<MOT` (qui se comporte comme des guillemets doubles) :

Fichier : quotes4.php

```
<?php
$nom = "Sami";
echo <<<FIN
Bonjour $nom,
  bienvenue chez Dar Zitouna.
FIN;
?>
```

Affichage obtenu

```
Bonjour Sami,
  bienvenue chez Dar Zitouna.
```

2.4 Concaténer : le point, pas le plus

`.` colle deux chaînes, `.=` ajoute à la fin d'une variable. Le `+` est **uniquement** arithmétique : `"5" + "3"` vaut `8`, et `5 . 3` vaut `"53"`.

Fichier : concat.php

```
<?php
$msg = "Bonjour";
$msg .= " ";
$msg .= "Rim";
echo $msg, "\n";
echo "5" + "3", " ", 5 . 3, "\n";
echo "Total : " . (2 + 3) . " DT\n";
?>
```

Affichage obtenu

```
Bonjour Rim
8 53
Total : 5 DT
```

2.5 Fonctions sur les chaînes

Fichier : chaines.php

```php
<?php
$ch = "Bonjour";
echo strlen($ch), "\n";
echo strtoupper($ch), " ", strtolower($ch), "\n";
echo ucfirst("sami"), " ", ucwords("dar zitouna"), "\n";
echo substr($ch, 0, 3), " ", substr($ch, -3), "\n";
echo strpos($ch, "j"), "\n";
echo str_replace("o", "0", $ch), "\n";
echo strrev($ch), " ", str_repeat("*", 5), "\n";
echo "[" . trim("  salut  ") . "]\n";
echo number_format(1234.5, 2, ",", " "), "\n";
echo htmlspecialchars("<b>Hi</b>"), "\n";
echo strlen("été"), " ", mb_strlen("été"), "\n";
$t = "dar zitouna";
echo substr_count($t, " ") + 1, " ", substr($t, strpos($t, " ") + 1), "\n";
?>
```

Affichage obtenu

```
7
BONJOUR bonjour
Sami Dar Zitouna
Bon our
3
B0nj0ur
ruojnoB *****
[salut]
1 234,50
&lt;b&gt;Hi&lt;/b&gt;
5 3
2 zitouna
```

Fonction	Rôle
`strlen($ch)`	nombre de caractères (en réalité d'octets : un é compte 2)
`mb_strlen($ch)`	vrai nombre de caractères avec les accents (UTF-8)
`strtoupper` / `strtolower` / `ucfirst` / `ucwords`	majuscules / minuscules / 1re lettre / 1re lettre de chaque mot
`substr($ch, début, longueur)`	extrait une partie ; début négatif = en partant de la fin
`strpos($ch, $cherche)`	position de la 1re occurrence (à partir de 0), `false` si absente
`str_replace($a, $b, $ch)`	remplace toutes les occurrences de `$a` par `$b`
`trim($ch)`	supprime les espaces au début et à la fin
`substr_count($ch, $x)`	nombre d'occurrences de `$x` dans la chaîne (nombre de mots = nombre d'espaces + 1)
`htmlspecialchars($ch)`	neutralise < > & " : à utiliser avant d'afficher une saisie d'utilisateur
`number_format($n, 2, ",", " ")`	met en forme un nombre (décimales, séparateurs)

À retenir

- `'...'` : texte brut. `"..."` : variables et `\n` interprétés.
- Choisir des guillemets PHP différents de ceux du HTML ; `.` pour concaténer.
- `'''` n'existe pas en PHP ; plusieurs lignes : retours à la ligne dans la chaîne ou heredoc `<<<MOT`.

3. Les constantes : define

Une **constante** est un nom auquel on associe une valeur qui ne changera plus pendant l'exécution. On l'utilise pour les paramètres de configuration (serveur, utilisateur, taux de TVA...). Contrairement à une variable, elle s'écrit **sans** `$`, par convention en MAJUSCULES, et elle est visible partout dans le script.

Fichier : define1.php

```
<?php
define("HOTE", "localhost");
define("TVA", 0.19);
const BASE = "darzitouna";
echo HOTE, " ", BASE, "\n";
echo "TVA : " . TVA * 100 . " %\n";
$ttc = 100 * (1 + TVA);
echo "TTC : $ttc DT\n";
if (defined("TVA")) {
    echo "TVA est définie\n";
}
echo "HOTE vaut " . HOTE . "\n";
echo "HOTE vaut HOTE\n";
?>
```

Affichage obtenu

```
localhost darzitouna
TVA : 19 %
TTC : 119 DT
TVA est définie
HOTE vaut localhost
HOTE vaut HOTE
```

	Variable	**Constante (define)**
Écriture	`$tva = 0.19;`	`define("TVA", 0.19);`
Utilisation	`$tva`	TVA (sans `$`, sans guillemets)
Dans une chaîne `"..."`	`"$tva"` est remplacée	`"TVA"` reste le mot TVA : concaténer avec `.`
Modification	possible à tout moment	**impossible** (un 2e `define` du même nom est ignoré avec un avertissement)
Portée	locale (voir fonctions)	visible partout, y compris dans les fonctions
Test d'existence	`isset($tva)`	`defined("TVA")`

À retenir

- `define("NOM", valeur);` : le nom est une **chaîne** en 1er argument ; ensuite on écrit `NOM` seul.
- Usage typique : paramètres de connexion à la base dans un fichier `config.php`, inclus par toutes les pages.
- Une constante ne s'interprète pas dans une chaîne `"..."` : `echo "Hôte : " . HOTE;`

4. Réutiliser du code : include et require

`include "fichier.php";` insère à cet endroit le contenu d'un autre fichier, comme si on l'avait écrit ici. On sépare ainsi la partie commune (connexion à la base, en-tête, pied de page) pour ne l'écrire qu'**une fois** dans le site.

Fichier : page.php

```
<?php
require "config.php";
?>
<html><body>
<?php include "entete.php"; ?>
<p>Base utilisée : <?= BASE ?></p>
<?php include "pied.php"; ?>
</body></html>
```

Affichage obtenu

```
<html><body>
<h1>Restaurant Dar Zitouna</h1>
<hr>
<p>Base utilisée : darzitouna</p>
<hr>
<p>Lycée Rafèha – 4ème SI</p>
</body></html>
```

Contenu de config.php (partagé par toutes les pages) :

```
<?php
define("HOTE", "localhost");
define("UTIL", "root");
define("MDP", "");
define("BASE", "darzitouna");
mysqli_report(MYSQLI_REPORT_OFF);
?>
```

Contenu de entete.php :

```
<h1>Restaurant Dar Zitouna</h1>
<hr>
```

Contenu de pied.php :

```
<hr>
<p>Lycée Rafèha – 4ème SI</p>
```

Instruction	Si le fichier est introuvable	Si le fichier est déjà inclus
`include`	avertissement (Warning), le script **continue**	le réinclut (risque : `define` en double)
`require`	erreur fatale, le script **s'arrête**	le réinclut
`include_once`	comme `include`	ne l'inclut **qu'une seule fois**
`require_once`	comme `require`	ne l'inclut **qu'une seule fois**

À retenir

- Fichier indispensable (connexion, configuration) → `require` ; élément optionnel (bandeau, menu) → `include`.
- Le chemin est relatif au script qui exécute l'instruction : `include "../config.php";` remonte d'un dossier.
- Un fichier inclus qui contient du PHP doit lui-même commencer par `<?php`.

Piège : Si `config.php` est inclus deux fois (directement et par un autre fichier), les `define` sont répétés : avertissement « Constant already defined ». Utiliser `require_once`.

5. Arrêter le script : die et exit

`die("message");` affiche le message puis **arrête immédiatement** l'exécution du script (`exit` est un synonyme). Tout ce qui est écrit après n'est jamais exécuté, y compris le reste de la page HTML. On l'emploie quand continuer n'a pas de sens : connexion impossible, paramètre manquant, accès sans passer par le formulaire.

Fichier : fiche.php

```
<?php
if (!isset($_GET["num"])) {
    die("Erreur : numéro manquant");
}
echo "Fiche du serveur " . $_GET["num"];
?>
```

Affichage : adresse fiche.php (sans paramètre)

```
Erreur : numéro manquant
```

Affichage – adresse fiche.php?num=3 – données envoyées : num=3

```
Fiche du serveur 3
```

5.1 La forme « ... or die(...) » avec mysqli_connect

`mysqli_connect` renvoie `false` en cas d'échec. L'opérateur `or` exécute `die` seulement si ce qui est à gauche vaut `false` :

```
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
```

Piège : Depuis PHP 8.1, `mysqli_connect` et `mysqli_query` **lèvent une exception** au lieu de renvoyer `false` : le `die` n'est alors jamais atteint et PHP affiche « Fatal error: Uncaught mysqli_sql_exception ». Pour retrouver le comportement classique, écrire une fois, dans `config.php` : `mysqli_report(MYSQLI_REPORT_OFF);` (c'est ce que fait notre `config.php`). Testé : sans cette ligne, un mauvais mot de passe donne l'exception ; avec, il affiche le message du `die`.

À retenir

- `die("message")` = afficher + arrêter. Le code qui suit n'est pas exécuté.
- Cas d'usage : `or die(...)` après `mysqli_connect` ; `if (!isset($_POST["valider"])) die(...)` pour refuser l'accès direct à une page de traitement.

6. Récupérer les données : GET et POST

Quand un visiteur envoie un formulaire (ou clique sur un lien avec paramètres), le navigateur transmet des couples **nom = valeur**. PHP les range dans deux tableaux associatifs **superglobaux**, disponibles partout : `$_GET` et `$_POST`. La **clé** du tableau est l'attribut `name` du champ HTML (pas son `id`), la valeur est ce que l'utilisateur a saisi, **toujours sous forme de chaîne**.

	GET	**POST**
Attribut du formulaire	`method="get"` (valeur par défaut)	`method="post"`
Tableau PHP	`$_GET["nom"]`	`$_POST["nom"]`
Où voyagent les données ?	dans l'**adresse** : `page.php?nom=Sami&age=17`	dans le **corps** de la requête (invisibles dans l'adresse)
Taille	limitée (environ 2000 caractères)	pratiquement illimitée
Mise en favori / partage du lien	oui	non
Mot de passe, données personnelles	**jamais** (visibles dans l'adresse et l'historique)	oui (avec HTTPS en vrai site)
Actualiser la page	sans question	le navigateur redemande la confirmation de renvoi

	GET	POST
À utiliser pour	recherche, filtre, navigation, lien vers une fiche	ajout, modification, suppression, connexion, inscription

6.1 Le formulaire et la page de traitement

Fichier : formulaire.html

```
<form action="traitement.php" method="post">
  Nom : <input type="text" name="nom">
  Salle :
  <select name="salle">
    <option value="Terrasse">Terrasse</option>
    <option value="Salon">Salon</option>
  </select>
  <input type="submit" name="valider" value="Envoyer">
</form>
```

Fichier : traitement.php

```
<?php
if (!isset($_POST["valider"])) {
    die("Accès interdit : passez par le formulaire");
}
$nom = $_POST["nom"];
$salle = $_POST["salle"];
echo "Bonjour " . htmlspecialchars($nom) . ", salle : " . $salle;
?>
```

Affichage : ouverture directe de la page, sans formulaire

```
Accès interdit : passez par le formulaire
```

Affichage – formulaire envoyé – données envoyées : nom=Rim, salle=Salon, valider=Envoyer

```
Bonjour Rim, salle : Salon
```

6.2 Données dans l'adresse : GET

Fichier : liste.html (liens avec paramètres)

```
<a href="fiche.php?num=3&nom=Karim">Voir Karim</a>
<a href="fiche.php?num=2&nom=Rim">Voir Rim</a>
```

Fichier : fiche2.php

```
<?php
$num = $_GET["num"];
$nom = $_GET["nom"];
echo "Serveur n°" . $num . " : " . $nom;
?>
```

Affichage – clic sur « Voir Karim » – données envoyées : num=3, nom=Karim

```
Serveur n°3 : Karim
```

6.3 Tester la présence d'une donnée : isset et empty

Fonction	Vrai quand…	Exemple
`isset($x)`	la variable ou la clé existe et n'est pas `NULL`	`isset($_POST["nom"])`
`empty($x)`	la valeur est « vide » : `""`, `"0"`, `0`, `NULL`, `false`, tableau vide, ou n'existe pas	`empty($_POST["nom"])`

Fichier : isset.php

```
<?php
$a = "0";
var_dump(isset($a), empty($a), isset($inexistant), empty($inexistant));
?>
```

Affichage obtenu

```
bool(true)
bool(true)
bool(false)
bool(true)
```

À retenir

- La clé de `$_GET` / `$_POST` = l'attribut **name** du champ HTML. Sans `name`, rien n'est envoyé.
- Une valeur reçue est **toujours une chaîne** : convertir avec `(int)` ou `(float)` avant de calculer.
- Un bouton radio ou une case **non cochés** ne sont **pas envoyés** : la clé n'existe pas → tester avec `isset`.
- Le bouton `submit` n'est envoyé que s'il a un `name` : `isset($_POST["valider"])` permet de savoir si le formulaire a été soumis.
- Avant d'afficher une saisie : `htmlspecialchars($_POST["nom"])`.

Piège : Lire `$_POST["nom"]` quand la clé n'existe pas (formulaire non envoyé, faute de frappe dans `name`) provoque : « Warning: Undefined array key "nom" ». Toujours tester avec `isset` avant de lire.

7. Types de données et conversion

PHP est **faiblement typé** : on ne déclare pas le type d'une variable, PHP le déduit de la valeur et convertit automatiquement quand une opération l'exige. Les principaux types (entier, réel, chaîne, booléen, tableau, NULL) sont décrits dans l'**Annexe A**. Ce chapitre insiste sur la **conversion**, indispensable pour les données de formulaire qui arrivent toujours en chaîne.

Conversion	Écriture	Exemple → résultat
vers entier	`(int)$x` ou `intval($x)`	`(int)"12abc"` → 12 ; `(int)"abc"` → 0 ; `(int)3.99` → 3 (troncature)
vers réel	`(float)$x` ou `floatval($x)`	`(float)"3.14"` → 3.14
vers chaîne	`(string)$x` ou `strval($x)`	`(string)45` → `"45"`
vers booléen	`(bool)$x`	`(bool)"0"` → false ; `(bool)""` → false ; `(bool)"abc"` → true
test avant conversion	`is_numeric($x)`	`is_numeric("12.5")` → true ; `is_numeric("12abc")` → false
connaître le type	`gettype($x)` ou `var_dump($x)`	`gettype("5")` → string

Fichier : conversion.php

```php
<?php
$a = "12";
$b = "3.5";
echo $a + $b, "\n";
echo $a . $b, "\n";
echo (int)"12abc", " ", intval("abc"), " ", (int)3.99, "\n";
echo (float)"3.14", " ", (string)45 . "7", "\n";
var_dump((bool)"0", (bool)"", (bool)"abc");
var_dump(is_numeric("12.5"), is_numeric("12abc"));
echo gettype($a), " ", gettype((int)$a), " ", gettype(1.5), " ", gettype(true), "\n";
echo round(3.456, 2), " ", floor(3.9), " ", ceil(3.1), "\n";
var_dump("10" == 10, "10" === 10);
?>
```

Affichage obtenu

```
15.5
123.5
12 0 3
3.14 457
bool(false)
bool(false)
bool(true)
bool(true)
bool(false)
string integer double boolean
3.46 3 4
bool(true)
bool(false)
```

`==` compare les valeurs après conversion (`"10" == 10` est vrai) ; `===` compare **valeur et type** (`"10" === 10` est faux). Pour un calcul sur une saisie : `$n = (int)$_POST["n"];` ou `$prix = (float)$_POST["prix"];`.

À retenir

- Les données de `$_GET` / `$_POST` sont des **chaînes** : les convertir avant un calcul ou une comparaison numérique.
- Conversion en entier = **troncature** (3.99 → 3) ; pour arrondir : `round()`.
- Valeurs considérées comme `false` : `false`, `0`, `0.0`, `""`, `"0"`, `[]`, `NULL` (voir Annexe A).

8. Conditions et boucles

Opérateur	Sens	Opérateur	Sens
==	égal (valeurs)	===	identique (valeur et type)
!= ou <>	différent	!==	non identique
< <= > >=	comparaisons	&& || !	et, ou, non
%	reste de la division	. .=	concaténation

Fichier : conditions.php

```
<?php
$note = 14;
if ($note >= 16) {
    echo "Très bien";
} elseif ($note >= 12) {
    echo "Bien";
} elseif ($note >= 10) {
    echo "Passable";
} else {
    echo "Insuffisant";
}
echo "\n";
$jour = 3;
switch ($jour) {
    case 1: echo "Lundi"; break;
    case 2: echo "Mardi"; break;
    case 3: echo "Mercredi"; break;
    default: echo "Autre jour";
}
echo "\n";
echo ($note >= 10) ? "Admis" : "Ajourné";
?>
```

Affichage obtenu

```
Bien
Mercredi
Admis
```

Dans un `switch`, chaque `case` se termine par `break` ; sans `break`, PHP continue sur le `case` suivant. L'expression `(condition) ? a : b` est une écriture courte de `if ... else`.

8.1 Les boucles

Fichier : boucles.php

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i * 3, " ";
}
echo "\n";
$n = 3;
while ($n > 0) {
    echo $n, " ";
    $n--;
}
echo "\n";
do {
    echo "exécuté au moins une fois\n";
} while (false);
$fruits = ["pomme", "poire"];
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i], "\n";
}
?>
```

Affichage obtenu

```
3 6 9 12 15
3 2 1
exécuté au moins une fois
pomme
poire
```

Les boucles servent aussi à **fabriquer du HTML** : ici une liste déroulante des années, et plus loin des boutons radio.

Fichier : select.php

```
<?php
echo "<select name='annee'>\n";
for ($a = 2026; $a >= 2023; $a--) {
    echo "<option value='$a'>$a</option>\n";
}
echo "</select>";
?>
```

Affichage obtenu

```
<select name='annee'>
<option value='2026'>2026</option>
<option value='2025'>2025</option>
<option value='2024'>2024</option>
<option value='2023'>2023</option>
</select>
```

À retenir

- `for` : nombre de tours connu ; `while` : tant qu'une condition est vraie ; `for` avec `count` : parcourir un tableau ; `do ... while` : au moins un tour.
- Attention à = (affectation) et == (comparaison) dans un `if`.

9. Tableaux et tableaux associatifs

Un **tableau** range plusieurs valeurs sous un seul nom. Dans un tableau **indexé**, on accède à une valeur par sa position (0, 1, 2…). Dans un tableau **associatif**, on y accède par une **clé** de notre choix (`"nom"`, `"salle"`…). `$_GET`, `$_POST`, et les lignes lues dans la base sont des tableaux associatifs.

9.1 Tableau indexé

Fichier : tab1.php

```
<?php
$notes = [12, 15, 9];
$notes[] = 18;
echo count($notes), " ", $notes[0], " ", $notes[3], "\n";
echo array_sum($notes) / count($notes), "\n";
echo max($notes), " ", min($notes), "\n";
sort($notes);
echo in_array(15, $notes) ? "15 présent" : "15 absent", "\n";
echo "Notes triées :";
for ($i = 0; $i < count($notes); $i++) {
    echo " " . $notes[$i];
}
echo "\n";
?>
```

Affichage obtenu

```
4 12 18
13.5
18 9
15 présent
Notes triées : 9 12 15 18
```

9.2 Tableau associatif

Fichier : tab2.php

```
<?php
$serveur = ["num" => 1, "nom" => "Sami", "salle" => "Terrasse"];
echo $serveur["nom"], "\n";
$serveur["salle"] = "Salon";
$serveur["tel"] = "71 000 000";
echo isset($serveur["tel"]) ? "tel connu" : "tel inconnu", "\n";
unset($serveur["tel"]);
$cles = array_keys($serveur);
for ($i = 0; $i < count($cles); $i++) {
    echo $cles[$i] . " = " . $serveur[$cles[$i]] . "\n";
}
var_dump(array_key_exists("age", $serveur));
?>
```

Affichage obtenu

```
Sami
tel connu
num = 1
nom = Sami
salle = Salon
bool(false)
```

9.3 Tableau de tableaux (comme les lignes d'une table)

Fichier : tab3.php

```
<?php
$menu = [
    ["plat" => "Couscous", "prix" => 22],
    ["plat" => "Lablabi",  "prix" => 9],
    ["plat" => "Brik",     "prix" => 4],
];
$total = 0;
for ($i = 0; $i < count($menu); $i++) {
    echo $menu[$i]["plat"] . " : " . $menu[$i]["prix"] . " DT\n";
    $total += $menu[$i]["prix"];
}
echo "Total : $total DT\n";
echo $menu[1]["plat"], "\n";
?>
```

Affichage obtenu

```
Couscous : 22 DT
Lablabi : 9 DT
Brik : 4 DT
Total : 35 DT
Lablabi
```

Fonction	Rôle
`count($t)`	nombre d'éléments
`$t[] = valeur;`	ajoute une valeur à la fin
`isset($t["k"])` / `array_key_exists("k", $t)`	la clé existe-t-elle ?
`in_array($v, $t)`	la valeur est-elle dans le tableau ?
`unset($t["k"])`	supprime un élément
`sort` / `rsort`	trie les valeurs (croissant / décroissant) ; `asort`, `ksort` : tri en gardant les clés / par clé
`array_sum` / `max` / `min`	somme / plus grande / plus petite valeur
`array_keys($t)` / `array_values($t)`	tableau des clés / des valeurs
`print_r($t)` / `var_dump($t)`	afficher le contenu pour déboguer (entourer de <pre> dans une page)

À retenir
- Tableau indexé : `for ($i = 0; $i < count($t); $i++)` et `$t[$i]`. Tableau associatif : `$cles = array_keys($t);` puis la même boucle `for` avec `$t[$cles[$i]]`.
- Dans une chaîne `"..."` : `"$t[0]"` ou `"{$t['nom']}"` (voir chapitre 2).

10. Les dates

10.1 date() et time()

`date(format, timestamp)` renvoie une date sous forme de **texte**, selon le **format** demandé. Le 2e argument est facultatif : sans lui, c'est la date et l'heure actuelles. Le **timestamp** est le nombre de secondes écoulées depuis le 1er janvier 1970 ; `time()` donne le timestamp actuel. Commencer ses scripts par `date_default_timezone_set("Africa/Tunis");`.

Fichier : date1.php

```
<?php
date_default_timezone_set("Africa/Tunis");
echo date("d/m/Y"), "\n";
echo date("d/m/Y H:i:s"), "\n";
echo date("Y-m-d"), "\n";
echo time() > 0 ? "timestamp positif" : "?", "\n";
?>
```

Affichage : exécuté le 14/11/2026 à 11 h (Tunis)

```
14/11/2026
14/11/2026 11:00:00
2026-11-14
timestamp positif
```

10.2 Les formats : majuscule ou minuscule, ça change tout

Chaque lettre du format a un sens précis et **la casse compte**. Le tableau complet est dans l'**Annexe B** ; voici les confusions les plus fréquentes (exemple pour le jeudi 5 mars 2026 à 14 h 07 min 09 s) :

Lettre	Affiche	Exemple	Lettre	Affiche	Exemple
`Y`	année sur 4 chiffres	2026	`y`	année sur 2 chiffres	26
`m`	mois sur 2 chiffres	03	`M`	mois abrégé (anglais)	Mar
`n`	mois sans zéro	3	`F`	mois complet (anglais)	March
`d`	jour sur 2 chiffres	05	`j`	jour sans zéro	5
`D`	jour abrégé (anglais)	Thu	`l`	jour complet (anglais)	Thursday
`H`	heure 00–23	14	`h`	heure 01–12	02
`i`	**minutes**	07	`m`	**mois** (pas les minutes !)	03
`s`	secondes	09	`A`	AM / PM	PM

Fichier : date2.php

```php
<?php
date_default_timezone_set("Africa/Tunis");
$t = mktime(14, 7, 9, 3, 5, 2026);
echo date("d/m/Y", $t), "\n";
echo date("d/m/y", $t), "\n";
echo date("Y-m-d H:i:s", $t), "\n";
echo date("D j M Y", $t), "\n";
echo date("l j F Y", $t), "\n";
echo date("h:i A", $t), "\n";
echo date("H:m:s", $t), "\n";
echo date("H:i:s", $t), "\n";
echo date('H\hi', $t), "\n";
?>
```

Affichage obtenu

```
05/03/2026
05/03/26
2026-03-05 14:07:09
Thu 5 Mar 2026
Thursday 5 March 2026
02:07 PM
14:03:09
14:07:09
14h07
```

Piège : `date("H:m:s")` affiche le **mois** à la place des minutes (14:**03**:09) : les minutes s'écrivent `i`. De même `M`, `D`, `l`, `F` donnent les noms **en anglais**. Pour écrire une lettre qui n'est pas un code (le `h` de « 14h07 »), la faire précéder d'un antislash : `'H\hi'` entre guillemets simples.

10.3 mktime, strtotime, checkdate : construire et tester des dates

Fichier : date3.php

```php
<?php
date_default_timezone_set("Africa/Tunis");
$t1 = strtotime("2026-03-05");
echo date("d/m/Y", $t1), "\n";
echo date("Y-m-d", strtotime("2026-03-05 +10 days")), "\n";
echo date("Y-m-d", strtotime("2026-01-31 +1 month")), "\n";
var_dump(checkdate(2, 30, 2026));
var_dump(strtotime("15/03/2026"));
$fr = "15/03/2026";
echo substr($fr, 6, 4) . "-" . substr($fr, 3, 2) . "-" . substr($fr, 0, 2) . "\n";
echo date("t", mktime(0, 0, 0, 2, 1, 2028)), " ", date("L", mktime(0, 0, 0, 1, 1, 2028)), "\n";
?>
```

Affichage obtenu

```
05/03/2026
2026-03-15
2026-03-03
bool(false)
bool(false)
2026-03-15
29 1
```

Fonction	Rôle	Ordre des arguments
`mktime(h, i, s, mois, jour, année)`	crée un timestamp	**heure, minute, seconde, MOIS, JOUR, ANNÉE** (le mois avant le jour !)
`strtotime("2026-03-05 +10 days")`	texte → timestamp	comprend `AAAA-MM-JJ`, `+1 week`, `-1 year`, `tomorrow`...
`checkdate(mois, jour, année)`	date valide ?	**MOIS, JOUR, ANNÉE**
`date_default_timezone_set(zone)`	fuseau horaire	`"Africa/Tunis"`

Piège : `strtotime("15/03/2026")` renvoie **false** : le format `jj/mm/aaaa` n'est pas compris (PHP lit `mm/jj/aaaa`). Pour convertir une date saisie en français, extraire le jour, le mois et l'année avec `substr` puis les recoller en `AAAA-MM-JJ`. Pour passer d'une date SQL à l'affichage français : `date("d/m/Y", strtotime($date_sql))`.

10.4 Écrire la date en français

Les formats `l` et `F` donnent l'anglais. En français, on range les noms dans des tableaux et on les indexe avec `w` (jour de la semaine, 0 = dimanche) et `n` (mois sans zéro).

Fichier : date4.php

```
<?php
date_default_timezone_set("Africa/Tunis");
$jours = ["dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"];
$mois = ["", "janvier", "février", "mars", "avril", "mai", "juin",
         "juillet", "août", "septembre", "octobre", "novembre", "décembre"];
$t = strtotime("2026-03-05");
echo $jours[date("w", $t)] . " " . date("j", $t) . " ";
echo $mois[date("n", $t)] . " " . date("Y", $t);
?>
```

Affichage obtenu

```
jeudi 5 mars 2026
```

10.5 Calculer l'âge avec PHP

L'âge n'est **pas** simplement « année actuelle − année de naissance » : si l'anniversaire de cette année n'est pas encore passé, il faut retirer 1. Une date saisie dans `<input type="date">` arrive au format `AAAA-MM-JJ`.

Fichier : age1.php

```
<?php
date_default_timezone_set("Africa/Tunis");
$naissance = "2008-05-12";
$an = substr($naissance, 0, 4);
$mois = substr($naissance, 5, 2);
$jour = substr($naissance, 8, 2);
$age = date("Y") - $an;
if (date("m") < $mois || (date("m") == $mois && date("d") < $jour)) {
    $age = $age - 1;
}
echo "Âge : $age ans\n";
$age2 = date("Y") - date("Y", strtotime($naissance));
if (date("md") < date("md", strtotime($naissance))) {
    $age2--;
}
echo "Âge (2e méthode) : $age2 ans\n";
?>
```

Affichage : exécuté le 14/11/2026

```
Âge : 18 ans
Âge (2e méthode) : 18 ans
```

Méthode 1 : on extrait l'année, le mois et le jour avec `substr` (positions 0, 5 et 8). **Méthode 2** : `date("md")` donne `1114` pour le 14 novembre ; comparer deux valeurs `mmjj` permet de savoir si l'anniversaire est passé. Les deux donnent le même âge.

Piège : Diviser un nombre de jours par 365 donne un âge **faux** : en ignorant les années bissextiles, on obtient 18 ans la veille des 18 ans.

Fichier : age2.php

```php
<?php
date_default_timezone_set("Africa/Tunis");
$naissance = "2008-11-15";
$faux = floor((time() - strtotime($naissance)) / (365 * 24 * 3600));
$age = date("Y") - date("Y", strtotime($naissance));
if (date("md") < date("md", strtotime($naissance))) {
    $age--;
}
echo "Méthode des 365 jours : $faux ans (faux)\n";
echo "Méthode correcte : $age ans\n";
?>
```

Affichage : exécuté le 14/11/2026 : l'anniversaire est demain

```
Méthode des 365 jours : 18 ans (faux)
Méthode correcte : 17 ans
```

10.6 Autres calculs sur les dates

Fichier : date5.php

```php
<?php
date_default_timezone_set("Africa/Tunis");
$d1 = "2026-03-05";
$d2 = "2026-12-25";
$jours = round((strtotime($d2) - strtotime($d1)) / 86400);
echo "Entre les deux dates : $jours jours, soit " . floor($jours / 7) . " semaines\n";
echo "Dans 30 jours : " . date("d/m/Y", strtotime("$d1 +30 days")) . "\n";
echo "Il y a 1 an : " . date("d/m/Y", strtotime("$d1 -1 year")) . "\n";
echo "Le 25 décembre est un " . date("l", strtotime($d2)) . "\n";
$naissance = "2008-05-12";
$prochain = date("Y") . date("-m-d", strtotime($naissance));
if ($prochain < date("Y-m-d")) {
    $prochain = (date("Y") + 1) . date("-m-d", strtotime($naissance));
}
$reste = round((strtotime($prochain) - strtotime(date("Y-m-d"))) / 86400);
echo "Prochain anniversaire : " . date("d/m/Y", strtotime($prochain));
echo ", dans $reste jours\n";
?>
```

Affichage : exécuté le 14/11/2026

```
Entre les deux dates : 295 jours, soit 42 semaines
Dans 30 jours : 04/04/2026
Il y a 1 an : 05/03/2025
Le 25 décembre est un Friday
Prochain anniversaire : 12/05/2027, dans 179 jours
```

Deux dates au format `AAAA-MM-JJ` peuvent se comparer **directement comme des chaînes** (`"2026-05-12" < "2026-11-14"` est vrai), car l'ordre alphabétique est l'ordre chronologique. La différence en jours se calcule sur les timestamps (secondes), divisés par 86400 (nombre de secondes dans un jour).

Besoin	Côté PHP	Côté SQL (série 8)
année / mois / jour	`date("Y")`, `date("m")`, `date("d")`	`YEAR()`, `MONTH()`, `DAY()`
différence en jours	`(strtotime($d2) - strtotime($d1)) / 86400`	`DATEDIFF(d1, d2)`
ajouter des jours	`date("Y-m-d", strtotime("$d +7 days"))`	`ADDDATE(d, 7)`
date du jour	`date("Y-m-d")`	`NOW()`

À retenir

- `date(format, timestamp)` : le format décide de tout ; `i` = minutes, `m` = mois, `Y` = 4 chiffres, `y` = 2 chiffres.
- `mktime(h, i, s, MOIS, JOUR, ANNÉE)` et `checkdate(MOIS, JOUR, ANNÉE)` : le mois avant le jour.
- Âge = différence des années, moins 1 si l'anniversaire n'est pas encore passé. Jamais « jours ÷ 365 ».
- `strtotime` comprend `AAAA-MM-JJ` mais pas `jj/mm/aaaa`.

11. MySQLi procédural : relier PHP à la base de données

PHP parle à MySQL/MariaDB avec les fonctions `mysqli_...`. Le schéma est toujours le même : **1. se connecter → 2. envoyer une requête SQL → 3. exploiter le résultat → 4. fermer**. Tous les exemples ci-dessous utilisent la base `darzitouna` de la série 8 et le fichier `config.php` du chapitre 4.

Fonction	Rôle	Retourne
`mysqli_connect(hote, util, mdp, base)`	ouvre la connexion (la base est facultative)	la connexion ; `false` en cas d'échec
`mysqli_select_db($cnx, base)`	choisit la base (si absente de `mysqli_connect`)	`true` / `false`
`mysqli_query($cnx, requête)`	envoie une requête SQL	`SELECT` : un **résultat** ; `INSERT`, `UPDATE`, `DELETE` : `true` ; `false` si erreur
`mysqli_num_rows($res)`	nombre de lignes d'un **SELECT**	un entier
`mysqli_fetch_row($res)`	ligne suivante, en tableau **indexé** (`$l[0]`, `$l[1]`...)	le tableau ; `NULL` quand il n'y a plus de ligne
`mysqli_fetch_array($res)`	ligne suivante, indexée **et** associative (`$l[0]` ou `$l["nom"]`)	le tableau ; `NULL` à la fin
`mysqli_affected_rows($cnx)`	lignes modifiées par `INSERT`, `UPDATE`, `DELETE`	un entier ; `0` si rien n'a changé ; `-1` si erreur
`mysqli_close($cnx)`	ferme la connexion	

11.1 Connexion

Fichier : connexion.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
echo "Connecté à la base " . BASE . "\n";
mysqli_close($cnx);
?>
```

Affichage obtenu

```
Connecté à la base darzitouna
```

Variante en deux temps : `$cnx = mysqli_connect(HOTE, UTIL, MDP);` puis `mysqli_select_db($cnx, BASE);`. La ligne `mysqli_set_charset($cnx, "utf8mb4")` évite les accents déformés.

11.2 SELECT : mysqli_query, mysqli_num_rows, mysqli_fetch_row

Fichier : serveurs.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
$res = mysqli_query($cnx, "SELECT num, nom, salle FROM SERVEUR");
echo mysqli_num_rows($res) . " serveur(s)\n";
while ($ligne = mysqli_fetch_row($res)) {
    echo $ligne[0] . " - " . $ligne[1] . " (" . $ligne[2] . ")\n";
}
mysqli_close($cnx);
?>
```

Affichage obtenu

```
4 serveur(s)
1 - Sami (Terrasse)
2 - Rim (Salon)
3 - Karim (Terrasse)
4 - Nour (Salon)
```

`mysqli_fetch_row` renvoie **une ligne à chaque appel** et avance à la suivante ; quand il n'y a plus de ligne, elle renvoie `NULL` (une valeur considérée comme fausse), ce qui arrête la boucle `while`. Les colonnes sont numérotées **dans l'ordre du SELECT** : `$ligne[0]` = `num`.

11.3 fetch_array : par numéro ou par nom de colonne

Fichier : desserts.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$res = mysqli_query($cnx, "SELECT plat, prix, qte FROM COMMANDE WHERE categorie = 'Dessert'");
while ($l = mysqli_fetch_array($res)) {
    echo $l["plat"] . " : " . $l["prix"] * $l["qte"] . " DT (colonne 0 = " . $l[0] . ")\n";
}
mysqli_close($cnx);
?>
```

Affichage obtenu

```
Makroudh : 30 DT (colonne 0 = Makroudh)
Assidat zgougou : 21 DT (colonne 0 = Assidat zgougou)
```

`mysqli_fetch_array` renvoie **les deux** : `$l[0]` et `$l["plat"]` désignent la même valeur. On préfère les noms de colonnes, plus lisibles. Le 2e argument choisit le format :

Fichier : fetch_types.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$sql = "SELECT num, nom FROM SERVEUR WHERE num = 2";
$res = mysqli_query($cnx, $sql);
echo json_encode(mysqli_fetch_array($res)), "\n";
$res = mysqli_query($cnx, $sql);
echo json_encode(mysqli_fetch_array($res, MYSQLI_ASSOC)), "\n";
$res = mysqli_query($cnx, $sql);
echo json_encode(mysqli_fetch_array($res, MYSQLI_NUM)), "\n";
mysqli_close($cnx);
?>
```

Affichage obtenu

```
{"0":"2","num":"2","1":"Rim","nom":"Rim"}
{"num":"2","nom":"Rim"}
["2","Rim"]
```

	`mysqli_fetch_row`	`mysqli_fetch_array`
Clés du tableau	numéros seulement	numéros **et** noms de colonnes (par défaut)
Écriture	`$l[1]`	`$l["nom"]` ou `$l[1]`
Option	aucune	`MYSQLI_ASSOC` (noms seulement), `MYSQLI_NUM` (numéros seulement), `MYSQLI_BOTH`
Fin des lignes	renvoie `NULL`	renvoie `NULL`

11.4 Une seule ligne attendue, ou aucune : mysqli_num_rows

Fichier : recherche.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$num = 3;
$res = mysqli_query($cnx, "SELECT nom, salle FROM SERVEUR WHERE num = $num");
if (mysqli_num_rows($res) == 0) {
    echo "Aucun serveur n°$num\n";
} else {
    $l = mysqli_fetch_array($res);
    echo $l["nom"] . " travaille en " . $l["salle"] . "\n";
}
$salle = "Salon";
$res = mysqli_query($cnx, "SELECT nom FROM SERVEUR WHERE salle = '$salle'");
echo mysqli_num_rows($res) . " serveur(s) en $salle\n";
$res = mysqli_query($cnx, "SELECT nom FROM SERVEUR WHERE nom = 'Zied'");
echo mysqli_num_rows($res) . " serveur nommé Zied\n";
mysqli_close($cnx);
?>
```

Affichage obtenu

```
Karim travaille en Terrasse
2 serveur(s) en Salon
0 serveur nommé Zied
```

Dans la requête, la requête SQL est entre **guillemets doubles** (pour que `$num` et `$salle` soient remplacés) ; une valeur de type **texte** est entourée d'**apostrophes** à l'intérieur (`'$salle'`), un **nombre** non (`= $num`). C'est la même règle que pour le HTML (chapitre 2).

11.5 INSERT, UPDATE, DELETE : mysqli_affected_rows

Fichier : maj.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_query($cnx, "INSERT INTO SERVEUR VALUES (5, 'Aya', 'Salon', '2025-09-01')");
echo "INSERT : " . mysqli_affected_rows($cnx) . " ligne(s)\n";
mysqli_query($cnx, "UPDATE COMMANDE SET prix = prix + 1 WHERE categorie = 'Boisson'");
echo "UPDATE : " . mysqli_affected_rows($cnx) . " ligne(s)\n";
mysqli_query($cnx, "UPDATE COMMANDE SET prix = prix WHERE num = 1");
echo "UPDATE sans changement : " . mysqli_affected_rows($cnx) . " ligne(s)\n";
mysqli_query($cnx, "DELETE FROM COMMANDE WHERE num > 10");
echo "DELETE : " . mysqli_affected_rows($cnx) . " ligne(s)\n";
$ok = mysqli_query($cnx, "INSERT INTO SERVEUR VALUES (5, 'Doublon', 'Salon', '2025-09-01')");
if (!$ok) {
    echo "Échec (clé en double) : affected_rows = " . mysqli_affected_rows($cnx) . "\n";
}
mysqli_close($cnx);
?>
```

Affichage obtenu

```
INSERT : 1 ligne(s)
UPDATE : 3 ligne(s)
UPDATE sans changement : 0 ligne(s)
DELETE : 2 ligne(s)
Échec (clé en double) : affected_rows = -1
```

Requête SQL	`mysqli_query` **retourne**	**Pour savoir « combien »**
`SELECT`	un résultat (ou `false` si erreur)	`mysqli_num_rows($res)`
`INSERT`, `UPDATE`, `DELETE`	`true` ou `false`	`mysqli_affected_rows($cnx)`

À retenir
- `mysqli_num_rows($res)` → après un **SELECT** ; `mysqli_affected_rows($cnx)` → après un **INSERT / UPDATE / DELETE** (on lui passe la connexion, pas le résultat).
- `while ($l = mysqli_fetch_row($res))` : une itération par ligne, arrêt automatique à la fin.
- Un `UPDATE` qui ne change aucune valeur (ou dont le `WHERE` ne trouve rien) renvoie `affected_rows = 0` : ce n'est pas une erreur.
- `mysqli_query` renvoie `false` si la requête est fausse (clé en double, faute de SQL…) : tester `if (!$ok)`.
- Une valeur texte dans une requête : entre apostrophes `'$x'` ; un nombre : sans apostrophes.

Piège : Passer à `mysqli_num_rows` le résultat d'une requête **échouée** (`false`) donne une erreur fatale : « Uncaught TypeError: mysqli_num_rows(): Argument #1 ($result) must be of type mysqli_result, false given ». Vérifier d'abord la requête SQL (guillemets, noms de colonnes). Une valeur reçue d'un formulaire insérée directement dans une requête expose le site à l'**injection SQL** : sujet de sécurité hors programme, mais à garder en tête.

12. Formulaires interactifs : boutons radio et calcul d'âge

Un formulaire HTML et sa page PHP forment un échange : l'utilisateur **choisit** ou **saisit**, PHP **reçoit, vérifie, calcule et répond**. Les programmes suivants regroupent le formulaire et son traitement dans **un seul fichier** : le formulaire s'envoie à la même page (`method="post"`, pas d'`action`) et PHP teste `isset($_POST["bouton"])` pour savoir si le formulaire vient d'être soumis.

12.1 Ce que PHP reçoit selon l'élément HTML

Élément HTML	Attributs importants	Ce que contient `$_POST`
Zone de texte, nombre, date	`name="nom"`	`$_POST["nom"]` : une **chaîne** (`type="date"` → `"2008-05-12"`)
Bouton radio	même `name` pour tout le groupe, `value` **différente** pour chaque bouton	`$_POST["civilite"]` = la `value` du bouton coché ; **clé absente** si aucun n'est coché
Case à cocher	`name="x"` (ou `name="plats[]"`)	la `value` si cochée, rien sinon ; avec `plats[]` : un **tableau**
Liste déroulante	`<select name="salle">` + `<option value="...">`	la `value` de l'option choisie
Bouton d'envoi	`<input type="submit" name="valider">`	`$_POST["valider"]` seulement si on a cliqué dessus

Boutons radio : les règles
- **Même** `name` pour tous les boutons radio d'un groupe : c'est ce qui les rend exclusifs (un seul coché à la fois).
- `value` **différente** pour chaque bouton : c'est elle, et non le texte affiché à côté, que PHP reçoit.
- `checked` pré-coche un bouton. Pour **conserver le choix** après l'envoi : `<?php if ($op == "+") echo "checked"; ?>`.
- Aucun bouton coché → `$_POST["nom_du_groupe"]` **n'existe pas** : toujours `isset` avant de lire.
- Envelopper `<label><input type="radio" ...> Texte</label>` rend le texte cliquable.

12.2 Programme 1 : civilité (bouton radio) et nom

Fichier : civilite.php

```
<?php
if (isset($_POST["envoyer"])) {
    if (isset($_POST["civilite"])) {
        $nom = htmlspecialchars($_POST["nom"]);
        echo "<p>Bonjour " . $_POST["civilite"] . " " . $nom . "</p>";
    } else {
        echo "<p>Veuillez choisir une civilité.</p>";
    }
}
?>
<form method="post">
  <p>Civilité :
    <input type="radio" name="civilite" value="Monsieur"> Monsieur
    <input type="radio" name="civilite" value="Madame"> Madame
  </p>
  <p>Nom : <input type="text" name="nom"></p>
  <input type="submit" name="envoyer" value="Valider">
</form>
```

Affichage – un bouton radio coché – données envoyées : civilite=Madame, nom=Ben Ali, envoyer=Valider (texte visible dans le navigateur)

```
Bonjour Madame Ben Ali
Civilité :
Monsieur
Madame
Nom :
```

Affichage – aucun bouton coché – données envoyées : nom=Ben Ali, envoyer=Valider (texte visible dans le navigateur)

```
Veuillez choisir une civilité.
Civilité :
Monsieur
Madame
Nom :
```

12.3 Programme 2 : calculatrice avec boutons radio (et mémorisation du choix)

Fichier : calculatrice.php

```
<?php
$a = "";
$b = "";
$op = "";
if (isset($_POST["calculer"])) {
    $a = $_POST["a"];
    $b = $_POST["b"];
    if (isset($_POST["op"]) && is_numeric($a) && is_numeric($b)) {
        $op = $_POST["op"];
        switch ($op) {
            case "+": $r = $a + $b; break;
            case "-": $r = $a - $b; break;
            case "*": $r = $a * $b; break;
            case "/":
                if ($b == 0) { $r = "impossible (division par zéro)"; }
                else { $r = $a / $b; }
                break;
        }
        echo "<p>$a $op $b = $r</p>";
    } else {
        echo "<p>Saisie incorrecte : deux nombres et une opération sont attendus.</p>";
    }
}
?>
<form method="post">
  <input type="text" name="a" value="<?= htmlspecialchars($a) ?>">
  <input type="radio" name="op" value="+" <?php if ($op == "+") echo "checked"; ?>> +
  <input type="radio" name="op" value="-" <?php if ($op == "-") echo "checked"; ?>> -
  <input type="radio" name="op" value="*" <?php if ($op == "*") echo "checked"; ?>> ×
  <input type="radio" name="op" value="/" <?php if ($op == "/") echo "checked"; ?>> ÷
  <input type="text" name="b" value="<?= htmlspecialchars($b) ?>">
  <input type="submit" name="calculer" value="=">
</form>
```

Affichage – 12 ÷ 4 – données envoyées : a=12, b=4, op=/, calculer== (texte visible dans le navigateur)

```
12 / 4 = 3
+
-
×
÷
```

Affichage – division par zéro – données envoyées : a=8, b=0, op=/, calculer== (texte visible dans le navigateur)

```
8 / 0 = impossible (division par zéro)
+
-
×
÷
```

Affichage – aucune opération choisie – données envoyées : a=8, b=2, calculer== (texte visible dans le navigateur)

```
Saisie incorrecte : deux nombres et une opération sont attendus.
+
-
×
÷
```

`is_numeric` vérifie que les saisies sont des nombres ; le `value="<?= ... ?>"` et le `checked` conditionnel **réaffichent** ce que l'utilisateur avait choisi. Le `switch` choisit l'opération d'après la `value` du bouton radio.

12.4 Programme 3 : calcul de l'âge (date de naissance + bouton radio)

L'utilisateur choisit sa civilité (bouton radio) et sa date de naissance (`type="date"`). PHP calcule : l'âge exact, majeur ou mineur, le jour de la semaine de naissance, le nombre de jours vécus et le **nombre de jours avant le prochain anniversaire**.

Fichier : age.php

```
<?php
date_default_timezone_set("Africa/Tunis");
$jours = ["dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"];
if (isset($_POST["calculer"]) && !empty($_POST["naissance"]) && isset($_POST["titre"])) {
    $naissance = $_POST["naissance"];
    $an = substr($naissance, 0, 4);
    $mois = substr($naissance, 5, 2);
    $jour = substr($naissance, 8, 2);
    $t = strtotime($naissance);
    $aujourdhui = strtotime(date("Y-m-d"));
    $age = date("Y") - $an;
    if (date("m") < $mois || (date("m") == $mois && date("d") < $jour)) {
        $age = $age - 1;
    }
    $vecu = round(($aujourdhui - $t) / 86400);
    $prochain = date("Y") . "-" . $mois . "-" . $jour;
    if ($prochain < date("Y-m-d")) {
        $prochain = (date("Y") + 1) . "-" . $mois . "-" . $jour;
    }
    $reste = round((strtotime($prochain) - $aujourdhui) / 86400);
    echo "<p>" . $_POST["titre"] . ", vous êtes né(e) le " . date("d/m/Y", $t);
    echo " (un " . $jours[date("w", $t)] . ").</p>";
    echo "<p>Âge : $age ans ";
    echo ($age >= 18) ? "(majeur)" : "(mineur)";
    echo "</p>";
    echo "<p>Jours vécus : $vecu</p>";
    echo "<p>Prochain anniversaire dans $reste jour(s)</p>";
} elseif (isset($_POST["calculer"])) {
    echo "<p>Choisissez une civilité et une date de naissance.</p>";
}
?>
<form method="post">
  <p>
    <input type="radio" name="titre" value="Monsieur"> Monsieur
    <input type="radio" name="titre" value="Madame"> Madame
  </p>
  <p>Date de naissance : <input type="date" name="naissance"></p>
  <input type="submit" name="calculer" value="Calculer l'âge">
</form>
```

Affichage – exécuté le 14/11/2026 – données envoyées : titre=Monsieur, naissance=2008-05-12, calculer=Calculer (texte visible dans le navigateur)

```
Monsieur, vous êtes né(e) le 12/05/2008 (un lundi).
Âge : 18 ans (majeur)
Jours vécus : 6760
Prochain anniversaire dans 179 jour(s)
Monsieur
Madame
Date de naissance :
```

Affichage – exécuté le 14/11/2026 – données envoyées : titre=Madame, naissance=2010-12-25, calculer=Calculer (texte visible dans le navigateur)

```
Madame, vous êtes né(e) le 25/12/2010 (un samedi).
Âge : 15 ans (mineur)
Jours vécus : 5803
Prochain anniversaire dans 41 jour(s)
Monsieur
Madame
Date de naissance :
```

Affichage – date manquante – données envoyées : titre=Madame, naissance=, calculer=Calculer (texte visible dans le navigateur)

```
Choisissez une civilité et une date de naissance.
Monsieur
Madame
Date de naissance :
```

À retenir

- `substr($naissance, 0, 4)`, `substr($naissance, 5, 2)` et `substr($naissance, 8, 2)` extraient l'année, le mois et le jour de la date `AAAA-MM-JJ` ; `strtotime` la convertit en timestamp ; `date("w", $t)` donne le jour de la semaine (0 = dimanche) qui indexe le tableau `$jours`.
- `$aujourdhui = strtotime(date("Y-m-d"))` : la date du jour **à minuit**, pour que les différences en jours tombent juste.
- `!empty($_POST["naissance"])` : un champ date laissé vide arrive comme chaîne vide.

12.5 Programme 4 : QCM avec boutons radio et score

Chaque question est un **groupe** de boutons radio (`q1`, `q2`, `q3`). Le tableau `$bonnes` contient la bonne réponse de chaque question, dans l'ordre, et une boucle `for` compare avec ce que PHP a reçu (`"q" . ($i + 1)` reconstruit le nom `q1`, `q2`…). Une question sans réponse n'est tout simplement pas dans `$_POST`.

Fichier : qcm.php

```
<?php
$bonnes = ["b", "a", "c"];
if (isset($_POST["corriger"])) {
    $score = 0;
    for ($i = 0; $i < count($bonnes); $i++) {
        $q = "q" . ($i + 1);
        if (isset($_POST[$q]) && $_POST[$q] == $bonnes[$i]) {
            $score++;
        }
    }
    echo "<p>Score : $score / " . count($bonnes) . "</p>";
}
?>
<form method="post">
  <p>1) Données d'un formulaire en POST :
    <input type="radio" name="q1" value="a"> $_GET
    <input type="radio" name="q1" value="b"> $_POST
    <input type="radio" name="q1" value="c"> $_SERVER</p>
  <p>2) Arrête le script avec un message :
    <input type="radio" name="q2" value="a"> die
    <input type="radio" name="q2" value="b"> echo
    <input type="radio" name="q2" value="c"> include</p>
  <p>3) Type d'une valeur reçue d'un formulaire :
    <input type="radio" name="q3" value="a"> int
    <input type="radio" name="q3" value="b"> bool
    <input type="radio" name="q3" value="c"> string</p>
  <input type="submit" name="corriger" value="Corriger">
</form>
```

Affichage – q3 non répondue – données envoyées : q1=b, q2=a, corriger=Corriger (texte visible dans le navigateur)

```
Score : 2 / 3
1) Données d'un formulaire en POST :
$_GET
$_POST
$_SERVER
2) Arrête le script avec un message :
die
echo
include
3) Type d'une valeur reçue d'un formulaire :
int
bool
string
```

12.6 Programme 5 : boutons radio fabriqués depuis la base de données

Les boutons radio ne sont pas écrits à la main : une boucle `while` lit les catégories avec `mysqli_fetch_row` et **génère** un bouton par catégorie. Après l'envoi, une 2e requête (avec `mysqli_fetch_array`) affiche les commandes de la catégorie choisie, leur nombre (`mysqli_num_rows`) et le chiffre d'affaires.

Fichier : categories.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
?>
<form method="post">
  <p>Catégorie :
<?php
$res = mysqli_query($cnx, "SELECT DISTINCT categorie FROM COMMANDE ORDER BY categorie");
while ($l = mysqli_fetch_row($res)) {
    echo "<input type='radio' name='cat' value='" . $l[0] . "'> " . $l[0] . " ";
}
?>
  </p>
  <input type="submit" name="afficher" value="Afficher">
</form>
<?php
if (isset($_POST["afficher"]) && isset($_POST["cat"])) {
    $cat = $_POST["cat"];
    $res = mysqli_query($cnx, "SELECT plat, prix, qte FROM COMMANDE WHERE categorie = '$cat'");
    echo "<p>" . mysqli_num_rows($res) . " commande(s) dans la catégorie $cat</p>";
    echo "<table border='1'>";
    $total = 0;
    while ($l = mysqli_fetch_array($res)) {
        $ca = $l["prix"] * $l["qte"];
        $total = $total + $ca;
        echo "<tr><td>" . $l["plat"] . "</td><td>$ca DT</td></tr>";
    }
    echo "</table>";
    echo "<p>Chiffre d'affaires : $total DT</p>";
}
mysqli_close($cnx);
?>
```

Affichage – catégorie Dessert – données envoyées : cat=Dessert, afficher=Afficher (texte visible dans le navigateur)

```
Catégorie :
Boisson Dessert Entrée Plat
2 commande(s) dans la catégorie Dessert
Makroudh | 30 DT
Assidat zgougou | 21 DT
Chiffre d'affaires : 51 DT
```

12.7 Programme 6 : choisir un format de date (boutons radio)

Cet exemple sert à **voir** l'effet de chaque lettre de format. Le tableau `$formats` alimente une boucle `for` qui fabrique les boutons radio. La date utilisée est fixe (5 mars 2026, 14 h 07 min 09 s) : remplacer `mktime(...)` par `time()` pour la date actuelle.

Fichier : formats.php

```
<?php
date_default_timezone_set("Africa/Tunis");
$formats = ['d/m/Y', 'Y-m-d', 'd/m/y', 'D j M Y', 'l j F Y',
            'H:i:s', 'H:m:s', 'H\hi', 'h:i A'];
if (isset($_POST["voir"]) && isset($_POST["format"])) {
    $f = $_POST["format"];
    echo "<p>Format " . htmlspecialchars($f) . " : ";
    echo date($f, mktime(14, 7, 9, 3, 5, 2026)) . "</p>";
}
?>
<form method="post">
<?php
for ($i = 0; $i < count($formats); $i++) {
    $f = $formats[$i];
    echo "<input type='radio' name='format' value='" . htmlspecialchars($f) . "'> $f<br>";
}
?>
  <input type="submit" name="voir" value="Voir">
</form>
```

Affichage – format H:m:s choisi – données envoyées : format=H:m:s, voir=Voir (texte visible dans le navigateur)

```
Format H:m:s : 14:03:09
d/m/Y
Y-m-d
d/m/y
D j M Y
l j F Y
H:i:s
H:m:s
H\hi
h:i A
```

Affichage – format H\hi choisi – données envoyées : format=H\hi, voir=Voir (texte visible dans le navigateur)

```
Format H\hi : 14h07
d/m/Y
Y-m-d
d/m/y
D j M Y
l j F Y
H:i:s
H:m:s
H\hi
h:i A
```

12.8 Programme 7 : cases à cocher (name="plats[]")

Fichier : plats.php

```
<form method="post">
  <input type="checkbox" name="plats[]" value="Couscous"> Couscous
  <input type="checkbox" name="plats[]" value="Lablabi"> Lablabi
  <input type="checkbox" name="plats[]" value="Brik"> Brik
  <input type="submit" name="commander" value="Commander">
</form>
<?php
if (isset($_POST["commander"])) {
    if (isset($_POST["plats"])) {
        $p = $_POST["plats"];
        echo count($p) . " plat(s) :";
        for ($i = 0; $i < count($p); $i++) {
            echo " " . $p[$i];
        }
    } else {
        echo "Aucun plat choisi";
    }
}
?>
```

Affichage – deux cases cochées – données envoyées : plats=[Couscous, Brik], commander=Commander (texte visible dans le navigateur)

```
Couscous
Lablabi
Brik
2 plat(s) : Couscous Brik
```

Avec le suffixe `[]` dans le `name`, PHP reçoit **un tableau** contenant les valeurs des cases cochées. Aucune case cochée → la clé `plats` n'existe pas.

13. Pour s'exercer

Les exercices (38 exercices progressifs avec jeux d'essai, sur tous les thèmes de ce document) sont dans le document séparé **Série d'exercices : PHP procédural**, accompagné de son corrigé. Les parties 1 à 8 de la série suivent l'ordre des chapitres 1 à 12 de ce cours.

Annexe A – Les types de données PHP

Type	Description	Exemples	Fonction de test
integer (`int`)	nombre entier	`42`, `-7`, `0`	`is_int($x)`
float (réel)	nombre à virgule (écrit avec un **point**)	`3.14`, `2.0`, `-0.5`	`is_float($x)`
string (`chaîne`)	suite de caractères	`"abc"`, `'12'`, `""`	`is_string($x)`
boolean (`bool`)	vrai ou faux	`true`, `false`	`is_bool($x)`
array (tableau)	plusieurs valeurs sous un nom	`[1, 2]`, `["a" => 1]`	`is_array($x)`
NULL	aucune valeur	`null`, variable non définie	`is_null($x)`
objet / résultat	retourné par `mysqli_connect` et `mysqli_query` ; on ne l'utilise qu'à travers les fonctions `mysqli_...`	`$cnx`, `$res`	`is_object($x)`

A.1 Valeurs considérées comme false

Dans une condition (`if`) ou avec `(bool)`, seules ces valeurs sont **fausses** : `false`, `0`, `0.0`, `""` (chaîne vide), `"0"` (le caractère zéro), `[]` (tableau vide), `NULL`. Tout le reste est **vrai**, y compris `"0.0"`, `"false"`, `" "` et `-1`.

A.2 Le type du résultat d'une opération

Fichier : types.php

```php
<?php
var_dump(10 / 4);
var_dump(10 / 5);
var_dump(7 % 3);
var_dump(intdiv(7, 2));
var_dump(2 ** 3);
var_dump("5" * "4");
var_dump(7 . "");
var_dump("12" + 3.5);
?>
```

Affichage obtenu

```
float(2.5)
int(2)
int(1)
int(3)
int(8)
int(20)
string(1) "7"
float(15.5)
```

La division `/` donne un entier quand le résultat est exact (`10 / 5` → `int(2)`), un réel sinon. `intdiv` donne le quotient entier, `%` le reste. Une opération arithmétique convertit les chaînes numériques en nombres ; le point `.` convertit en chaîne.

Annexe B – Les formats de la fonction date()

Exemples calculés pour le **jeudi 5 mars 2026 à 14 h 07 min 09 s** (fuseau `Africa/Tunis`). **La casse compte** : `d` ≠ `D`, `m` ≠ `M`, `y` ≠ `Y`, `h` ≠ `H`, `a` ≠ `A`. Les noms de jours et de mois sont en **anglais**.

B.1 Jour

Code	Signification	Exemple
`d`	jour du mois sur **2 chiffres** (01 à 31)	05
`j`	jour du mois **sans zéro** (1 à 31)	5
`D`	jour de la semaine, 3 lettres, en anglais (Mon … Sun)	Thu
`l`	jour de la semaine complet, en anglais (Monday …)	Thursday
`N`	numéro du jour de la semaine : 1 = lundi … 7 = dimanche	4
`w`	numéro du jour de la semaine : 0 = dimanche … 6 = samedi	4
`z`	numéro du jour dans l'année (0 à 365)	63
`W`	numéro de la semaine dans l'année (ISO)	10

B.2 Mois

Code	Signification	Exemple
`m`	mois sur **2 chiffres** (01 à 12)	03
`n`	mois **sans zéro** (1 à 12)	3
`M`	mois abrégé en anglais (Jan … Dec)	Mar
`F`	mois complet en anglais (January …)	March
`t`	nombre de jours du mois (28 à 31)	31

B.3 Année

Code	Signification	Exemple
`Y`	année sur **4 chiffres**	2026
`y`	année sur **2 chiffres**	26
`L`	année bissextile : 1 = oui, 0 = non	0

B.4 Heure, minutes, secondes

Code	Signification	Exemple
`H`	heure **00 à 23**, sur 2 chiffres	14
`G`	heure 0 à 23, sans zéro	14
`h`	heure **01 à 12**, sur 2 chiffres	02
`g`	heure 1 à 12, sans zéro	2
`A`	AM ou PM	PM
`a`	am ou pm	pm
`i`	**minutes** (00 à 59) – attention : pas `m` !	07
`s`	secondes (00 à 59)	09
`U`	timestamp (secondes depuis le 1/1/1970)	1772716029
`e`	fuseau horaire	Africa/Tunis

B.5 Combinaisons usuelles

Format	Résultat	Usage / remarque
`d/m/Y`	05/03/2026	date française
`d-m-Y`	05-03-2026	variante avec tirets
`Y-m-d`	2026-03-05	format **SQL** (colonnes DATE, `type="date"`)
`Y-m-d H:i:s`	2026-03-05 14:07:09	format SQL DATETIME
`d/m/y`	05/03/26	année sur 2 chiffres
`j/n/Y`	5/3/2026	sans zéros devant
`D j M Y`	Thu 5 Mar 2026	abréviations anglaises
`l j F Y`	Thursday 5 March 2026	noms complets anglais
`H:i`	14:07	heure 24 h
`h:i A`	02:07 PM	heure 12 h avec AM/PM
`H:m:s`	14:03:09	**erreur classique** : `m` = mois, pas minutes
`'H\hi'`	14h07	le h est précédé de `\` : lettre affichée telle quelle
`'\L\e d/m/Y'`	Le 05/03/2026	texte littéral échappé lettre par lettre

B.6 Récapitulatif des confusions fréquentes

Confusion	Règle pour ne pas se tromper
`m` ou `i` ?	`m` = **m**ois ; `i` = minutes (pas de moyen mnémotechnique : à apprendre)
`Y` ou `y` ?	`Y` majuscule = année **entière** (2026) ; `y` = 2 chiffres (26)
`d` ou `j` ?	`d` = 2 chiffres avec zéro (05) ; `j` = sans zéro (5)
`m` ou `n` ?	`m` = 2 chiffres (03) ; `n` = sans zéro (3)
`m` ou `M` ou `F` ?	`m` = numéro ; `M` = nom abrégé anglais ; `F` = nom complet anglais
`d` ou `D` ou `l` ?	`d` = numéro ; `D` = nom abrégé anglais ; `l` (L minuscule) = nom complet anglais
`H` ou `h` ?	`H` majuscule = 00 à **23** ; `h` = 01 à **12** (à accompagner de `A`)
`w` ou `N` ?	`w` : dimanche = 0 ; `N` : lundi = 1, dimanche = 7
Lettre à afficher telle quelle	la précéder de `\` (`\h`) ou utiliser des guillemets simples

Annexe C – Récapitulatif MySQLi procédural

Fonction	Rôle	Valeur retournée
`mysqli_connect($hote, $util, $mdp, $base)`	ouvrir la connexion	connexion ; `false` si échec
`mysqli_select_db($cnx, $base)`	choisir la base	`true` / `false`
`mysqli_set_charset($cnx, "utf8mb4")`	définir l'encodage (accents)	`true` / `false`
`mysqli_query($cnx, $sql)`	exécuter une requête SQL	SELECT : résultat ; INSERT/UPDATE/DELETE : `true` ; erreur : `false`
`mysqli_num_rows($res)`	nombre de lignes du résultat d'un SELECT	entier
`mysqli_fetch_row($res)`	ligne suivante, tableau indexé `[0]`, `[1]`...	tableau ou `NULL` à la fin
`mysqli_fetch_array($res)`	ligne suivante, indexé **et** associatif	tableau ou `NULL` à la fin
`mysqli_fetch_array($res, MYSQLI_ASSOC)`	ligne suivante, noms de colonnes seulement	tableau ou `NULL`
`mysqli_fetch_array($res, MYSQLI_NUM)`	ligne suivante, numéros seulement	tableau ou `NULL`
`mysqli_affected_rows($cnx)`	nombre de lignes modifiées par INSERT/UPDATE/DELETE	entier ; 0 = rien changé ; -1 = erreur
`mysqli_error($cnx)`	texte de la dernière erreur SQL (utile pour déboguer)	chaîne
`mysqli_close($cnx)`	fermer la connexion	`true`

Modèle de page MySQLi (à retenir)

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");

$res = mysqli_query($cnx, "SELECT ... FROM ... WHERE ...");   // 1) requête
if (mysqli_num_rows($res) == 0) {                             // 2) résultat vide ?
    echo "Aucun résultat";
}
while ($ligne = mysqli_fetch_array($res)) {                   // 3) une itération par ligne
    echo $ligne["colonne"];
}

mysqli_query($cnx, "INSERT / UPDATE / DELETE ...");           // modification
echo mysqli_affected_rows($cnx) . " ligne(s) modifiée(s)";

mysqli_close($cnx);
?>
```

Annexe D – Messages d'erreur fréquents

Messages relevés avec PHP 8.3 (les numéros de ligne et les chemins varient).

Message	Cause probable	Remède
`Undefined variable $x`	variable jamais affectée, ou faute de frappe dans son nom	vérifier l'orthographe et l'ordre des instructions
`Undefined array key "nom"`	clé absente de `$_POST`, `$_GET` ou d'un tableau : formulaire non envoyé, `name` mal écrit, radio non coché	`isset($_POST["nom"])` avant de lire ; contrôler les attributs `name`
`Constant TVA already defined`	deux `define` du même nom (fichier inclus plusieurs fois)	`require_once` ; un seul `define` par constante
`A non-numeric value encountered`	calcul sur une chaîne non numérique (`"3 pommes" + 2`)	tester avec `is_numeric`, convertir avec `(int)` / `(float)`
`Failed opening required 'x.php'`	chemin du fichier inclus faux	corriger le chemin (relatif au script qui exécute `require`)
`Parse error: syntax error, unexpected ...`	`;` oublié, guillemet ou parenthèse non fermé, $ oublié	relire la ligne indiquée **et la précédente**
`mysqli_num_rows(): Argument #1 ($result) must be of type mysqli_result, false given`	la requête a échoué (`false`) : erreur SQL, mauvais guillemets	afficher la requête, tester `mysqli_error($cnx)`
`Uncaught mysqli_sql_exception`	PHP ≥ 8.1 : une erreur MySQL lève une exception, `or die` n'est jamais atteint	`mysqli_report(MYSQLI_REPORT_OFF);` dans `config.php`
Page blanche	erreurs masquées par la configuration du serveur	activer `display_errors` (php.ini) en phase de développement
Accents déformés (Ã©)	encodage	fichiers en UTF-8, `mysqli_set_charset($cnx, "utf8mb4")`, `<meta charset="UTF-8">`

Enseignant : Aymen Essouyah – Lycée Rafèha – 4ème SI